ZaturoBot

Your personal Linux assistant.
Native clients. Bounded services. Private control.

INITIAL NATIVE RELEASE

Owner-first execution. Evidence-first reporting.

Zaturo gold

#E5B900

Primary action / focus / meaningful emphasis

Foundation black

#0B0B0D

Background / private panel / visual calm

Contrast white

#FFFFFF

Core text / original mark / clear hierarchy

TYPOGRAPHY AND RHYTHM

Inter — Interface target

JetBrains Mono — Code / logs target

This PDF embeds Noto Sans and Noto Sans Mono as verified local fallbacks. Inter and JetBrains Mono are not bundled or claimed installed.

ACCESSIBLE USE

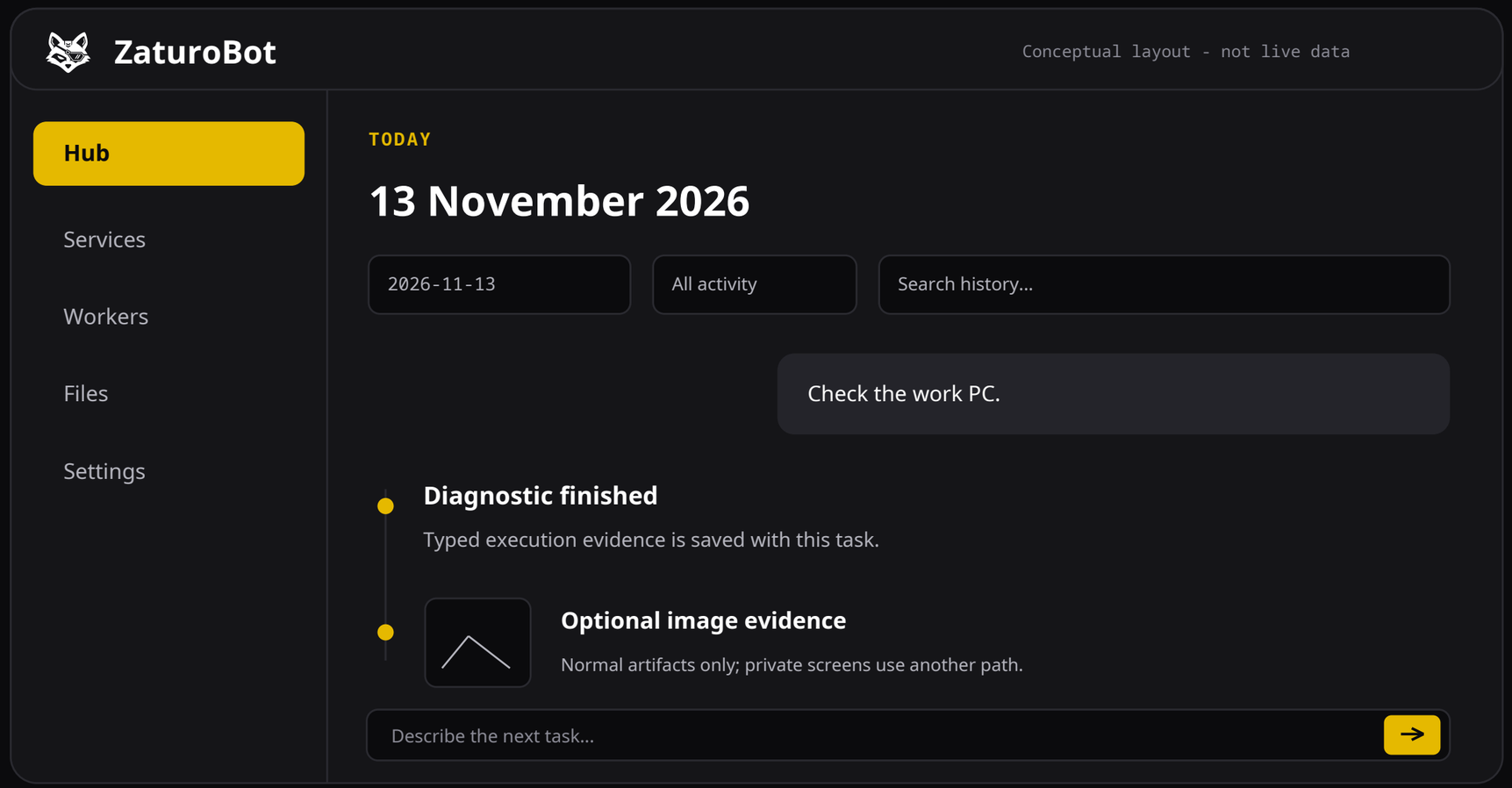
White / black 19.66:1

Gold / black 10.55:1

Use dark text on gold buttons. White on gold is only 1.86:1 and is unsuitable for small functional text. Space on a 4 / 8 / 12 / 16 / 24 / 32 / 48 scale.

One hub. The day in focus.

Chat and an activity feed share a durable history. Today is the initial view; date, type and search filters retrieve earlier work.



TODAY BY DEFAULT

Persist UTC; present the requested local day using the client offset. Initial owner zone: America/Cuiaba.

HISTORY REMAINS AVAILABLE

Changing the day is a query, not deletion. Completed tasks and ordinary evidence stay linked.

COMPLETION FOLLOWS EVIDENCE

Admitted video work and finished video work are different events. The feed reports actual job transitions.

PRIVATE VIEWING IS SEPARATE

Sensitive phone images never become feed thumbnails, model context or ordinary stored artifacts.

A saved chat message is not an executed task.

Only explicitly supported typed actions are dispatched. Other messages remain in history with an honest, unexecuted status. The owner interface defaults to Brazilian Portuguese; labels here are in English for design review.

Keep coordination small. Execute at the edge.

Linux uses Rust + Slint. Android uses Kotlin. Both reach a private hub; notebook services own computation, credentials and USB hardware.

OWNER CLIENTS

Linux desktop

Rust + Slint

Android app

Kotlin / Android only

Scoped app credentials
No provider key in a UI bundle

SSH 22 → outbound

ORACLE VPS / 1 GB

Rust hub + SQLite

Tasks, feed and worker metadata

Authentication, leases and audit

Disk: ordinary encrypted file spool

RAM: expiring private ciphertext

SSH 22 ← outbound

LINUX NOTEBOOK / 22 GB

Rust worker

Outbound task connection

Local journal and safe adapters

Keys and account profiles stay local

age encryption before file upload

INDEPENDENT HEAVY JOB

YoutubeBot actor

Separate systemd user cgroup.
Video / browser / upload workload.

USB phone

A DELIBERATE RESOURCE BOUNDARY

The VPS does not become a remote workstation.

No video rendering, browser, ADB server or account credentials run there. Heavy video jobs are independent of the lightweight worker's 512 MiB memory ceiling. Private SSH host checks stay enabled; home-router inbound ports stay closed.

PROTOCOL 2 / CAPABILITY-GATED SERVICES / HUB SCHEMA 5 / WORKER SCHEMA 6

Slint Rust UI https://docs.slint.dev/latest/docs/slint/guide/getting-started/

Preserve the pipeline. Separate the identity.

The account manager and service boundary surround the existing extension, video generation and uploader. Original local source files remain preserved.

01

Choose an account

Use the real Google browser consent window; no embedded password form.

02

Bind the channel

PKCE + state protect authorization. Verify the actual YouTube channel ID.

03

Keep profiles apart

Separate Google identity, channel bindings and browser profile directories.

04

Manage the library

Organize titles, topics, runs, schedule slots and bounded artifact summaries.

EXISTING PRODUCTION PATH / BEHAVIOR RETAINED

Browser extension → **Script + audio** → **Video rendering** → **Upload + schedule**

DURABLE BACKGROUND LIFECYCLE

accepted → **running** → **completed**

Durable admission

Actual actor started

Scheduled-item evidence

failed

Known failure

uncertain

Status-only reconciliation

INDEPENDENT ACTOR LIMITS

12G / 16G

MemoryHigh / MemoryMax

200% CPU / 12 h / no restart

Never report admission as a finished video. A lost response queries the same operation; it never blindly submits again.

Google desktop OAuth https://developers.google.com/identity/protocols/oauth2/native-app

Private screens take a different route.

A screenshot can contain authentication material. The user authorizes a finite view; pixels never enter ordinary history, a provider prompt or the normal file spool.

01

Owner grant

Exact phone and viewer.
Explicit confirmation.
Immediate or 5-120 s watch.

→

02

USB capture

Authorized, unlocked phone.
PNG in protected memory.
Serial + revocable lease.

→

03

age encryption

Registered viewer recipient.
Private key at endpoint.
No OCR. No LLM.

→

04

RAM relay

At most two sessions.
Cipher expires in <=60 s.
No SQL or image files.

→

05

Private viewer

Linux desktop decrypts.
Consume once; clear on
expiry, close or focus loss.

History stores metadata only.

The acceptance contract rejects extra image fields, code strings and arbitrary private captions, even when they fit inside a small JSON payload.

```
{
  "session_id": "00000000-0000-4000-8000-000000000000",
  "delivery": "ephemeral_encrypted_memory",
  "max_view_seconds": 60
}
```

ANDROID APP: CHAT + FEED / PRIVATE AGE IMAGE VIEWER: LINUX DESKTOP

ADB screen capture https://developer.android.com/tools/adb
Secure windows https://developer.android.com/reference/android/view/WindowManager.LayoutParams#FLAG_SECURE

AN IMAGE CHANGE IS NOT AN SMS DETECTOR

A clock, animation or app switch can trigger the finite watch. It returns at most the first changed screen; an unchanged interval ends normally. No notification harvesting or authentication-code extraction is implemented.

RESPECT ANDROID PROTECTIONS

The owner prepares the SMS screen and performs any required unlock or USB RSA consent. No root, default-SMS-handler change, lock-screen bypass or FLAG_SECURE workaround is introduced.

Deploy additively. Verify the actual outcome.

The work PC is already enrolled on 0.3. The 0.4 update preserves its private keys, identity and journal; source changes and local assets follow separate release paths.

API SURFACE

`/v1/chat`

Durable request + explicit typed actions

`/v1/feed`

Day, type, search and bounded history pages

`/v1/services`

Worker capability inventory

`/v1/device/activity`

Actual background service transitions

`/v1/private-views`

Owner grant + one-time, encrypted RAM delivery

`/v1/device/transfers`

Resumable ordinary age-encrypted files

FILE AND IMAGE LIMITS

2 GiB file / 8 GiB spool / 24 h

Ordinary transfer defaults; 4 MiB resumable chunks. Quota is released after expired ciphertext is actually removed.

8 MiB PNG / 2 private sessions

Cipher cap: 8 MiB + 64 KiB. One-time grant and receipt, active expiry and no-store responses; no sensitive screenshot disk cache.

LOCAL ASSET MIGRATION / SEPARATE PROFILE

12,455 captured local files: 8.90 GB source, 7.71 GB outer age cipher. Eight parts of at most 1 GiB are encrypted again by the file transport. Temporary VPS policy: 16 GiB spool and seven-day retention.

IMPLEMENTED AND TESTED IN CODE

Rust task/feed/service contracts; Linux native client; Kotlin Android debug build; bounded adapters; replay fencing; synthetic-device privacy tests. Native binaries and APK are build artifacts, not proof of real account or hardware consent.

PHYSICAL ACCEPTANCE STILL REQUIRED

Google browser consent, the exact USB phone, protected-screen behavior and real video generation/upload have not been exercised. Verify them on the work PC before claiming production success. Rebuildable caches, models and virtual environments are excluded from migration.

NO CREDENTIALS IN GIT / NO PUBLIC ADB / NO BLIND SIDE-EFFECT REPLAY